CHAPTER 4

Collections, Blocks, and Iterators

Most real programs deal with collections of data: the people in a course, the songs in your playlist, the books in the store. Ruby comes with two classes that are commonly used to handle these collections: *arrays* and *hashes*. A Ruby hash is a key/value pair, equivalent to a Python dictionary, a Java Map, or a JavaScript object. Mastery of these two classes, and their large interfaces, is an important part of being an effective Ruby programmer.

But it isn't just these two classes that give Ruby its power when dealing with collections. Ruby also has a block syntax that lets you encapsulate chunks of code. When paired with collections, these blocks can build powerful iterator constructs. In this chapter, we'll look at the two collection classes as well as blocks and iterators.

Arrays

The class Array holds a collection of object references. Each object reference occupies a position in the array, identified by an integer index. You can create arrays by using literals or by explicitly creating an Array object. A literal array is a comma-delimited list of objects between square brackets:

```
a = [3.14159, "pie", 99]
a.class   # => Array
a.length  # => 3
a[0]      # => 3.14159
a[1]      # => "pie"
a[2]      # => 99
a[3]      # => nil
```

You can create an empty array with either [] or by directly calling Array.new:

```
b = Array.new
b.class   # => Array
b.length  # => 0
b[0] = "second"
b[1] = "array"
b         # => ["second", "array"]
```

As the example shows, array indices start at zero. Index an array with a non-negative integer, and it returns the object at that position or returns nil if nothing is there. Index an array with a negative integer, and it counts from the end, with -1 being the last element of the array.